

Object gericht-programmeren — Synthax Java

Ruben Ryckaert

17 januari 2026

Samenvatting

Deze samenvattingen zijn beschikbaar op GitHub waar je kunt bijdragen aan verbeteringen: <https://github.com/Eggmansmile/Samenvattingen-Ku-leuven-Industriële-ingenieurs>

Je bent helemaal vrij en ik raad je zelf aan om info toe te voegen aan de documenten als je de uitleg niet goed vindt. Je leert je vak en je raakt dan ook bekend met LaTeX wat handig is voor later.

Je mag voor het examen een **handgeschreven 4-pagina's A4-formularium meenemen** met daarop alles wat je nodig hebt. Dit document moet je tonen wat de belangrijkste dingen zijn die je moet weten voor het examen.

W3schools heeft een goede Java tutorial: <https://www.w3schools.com/java/> Bekijk die zeker als je niet weet wat alles doet. Dit is een goed beginpunt.

1 Basis java

1.1 Import statements

Om klassen van andere packages te gebruiken, moeten deze eerst geïmporteerd worden met een import statement.

```
</> Import utils

1  import java.util.*; // importeert alle klassen van de package java.util
2  import java.util.ArrayList; // importeert enkel de ArrayList klasse
```

1.2 Typische Java klasse structuur & waarden

Een typische Java klasse heeft de volgende structuur:

```
</> Hoe java doc starten?

1  package packageNaam; // optioneel
2
3  import package1.Klasse1; // optioneel
4  import package2.*; // optioneel
5
6  public class KlasseNaam { // klasse definitie
7
8      // opstellen van variabelen (attributen)
9      private int attribuut1;
10     private String attribuut2;
11
12     // constructor (Je creeert een instance van een klasse met een constructor)
13     public KlasseNaam(int attribuut1, String attribuut2) {
14         this.attribuut1 = attribuut1;
15     }
16 }
```

```

15         this.attribuut2 = attribuut2;
16     }
17
18     // methoden
19     public void methode1() {
20         // methode implementatie
21     }
22
23     public int methode2(String parameter) {
24         // methode implementatie
25         return 0;
26     }
27
28     #alle soorten characters:
29     int #gehele getallen
30     int [] #single dim array
31     int [] [] #multi dim arrays
32
33     String #tekst
34     String [] #array van teksten
35
36     float #decimale getallen
37
38     boolean #true of false
39
40     char #enkel character
41
42     double #decimale getallen (anders dan float)
43     double [] #array van double
44
45     //belangrijke na elke lijn ;
46
47 }

```

1.3 Methoden

een methode zit in je klasse. Hiermee kun je functionaliteit toevoegen aan je klasse.

```

</> methode voorbeeld
1
2 public int /**return type*/ methodeOptelling(int parameter1, int parameter2)
3 {
4     int resultaat = parameter1 + parameter2;
5     return resultaat; // geeft het resultaat terug
6
7 }
8
9 een methode met return type 'void' geeft niets terug:
10 public void methodePrint(String tekst)
11 {
12     System.out.println(tekst); // print de tekst naar de console
13 }
14
15 Je kunt alle characters retourneren die je wilt.

```

2 If, for en while

2.1 If statements

```
</> if statement

1  if (voorwaarde) {
2      // code die uitgevoerd wordt als de voorwaarde waar is
3  } else if (andereVoorwaarde) {
4      // code die uitgevoerd wordt als de andere voorwaarde waar is
5  } else {
6      // code die uitgevoerd wordt als geen van de voorwaarden waar is
7  }

8
9  if en else zijn een sort scan, dus eerst wordt if gedaan en als
10 dat niet waar is, gaat die naar else if en zo verder.
11 Moest je nu gewoon if statements gebruiken, dan worden ze allemaal
12 uitgevoerd.
13
14 Dit zijn belangrijke voorwaarden:
15 == // gelijk aan
16 != // niet gelijk aan
17 > // groter dan
18 < // kleiner dan
19 >= // groter dan of gelijk aan
20 <= // kleiner dan of gelijk aan
21 && // en
22 || // of
```

2.2 For loops

```
</> for loop

1  for (init; voorwaarde; increment) {
2      // code die herhaald wordt
3  }

4
5  // voorbeeld: print getallen van 0 tot 9
6  for (int i = 0; i < 10; i++) // i begint bij 0, zolang i kleiner is dan 10, verhoog i met 1
7  //vergeet de ; niet tussen de statements (init, voorwaarde, increment)
8  {
9      System.out.println(i); //output is 0 1 2 3 4 5 6 7 8 9
10 }
```

2.3 While loops

While loops zijn gelijkaardig aan for loops, maar ze zijn meer geschikt voor situaties waarin je niet weet hoeveel iteraties je nodig hebt.

```
</> while loop

1  int i = 0; // initialisatie
2  while (i < voorwaarde) // zolang i kleiner is dan de voorwaarde
3  {
4      // code die herhaald wordt zolang de voorwaarde waar is
5      i++; // vergeet niet i aan te passen om een oneindige loop te vermijden
6      i--; // of i verlagen
```

```
7 }
```

2.4 Arrays

een array is wiskundig een vector. een array van strings zijn gewoon meerdere teksten in 1 variabele.

```
</> array

1 // array van gehele getallen met 5 elementen
2 int[] getallen = new int[5];
3
4 // array van strings met 3 elementen
5 String[] namen = new String[3]; // een array zijn lengte is vast. pas op want dit is niet de index.
6
7 // array initialisatie met waarden
8 int[] cijfers = {90, 85, 78, 92, 88};
9
10 // toegang tot array elementen
11 int eersteGetal = getallen[0]; // eerste element (index 0)
12 String eersteNaam = namen[0]; // eerste element (index 0)
13
14 //belangrijke array functies
15 int lengte = cijfers.length; // lengte van de array
16 int laatsteIndex = cijfers.length - 1; // laatste index van de array
17 int middelsteIndex = cijfers.length / 2; // middelste index van de array
18 int
19 // itereren over een array
20 for (int i = 0; i < cijfers.length; i++) {
21     System.out.println(cijfers[i]);
22 }
23
24 je hebt nog een foreach loop. Deze moet je zeker kennen voor ArrayLists maar
25 is ook hier handig.
26
27 for (int cijfer : cijfers) //zorg dat je de : gebruikt in plaats van ;
28 // je zegt dus voor elk cijfer (die gebruik je in de loop) in cijfers (de array)
29 {
30     System.out.println(cijfer);
31 }
```

2.5 Multi dim arrays

```
</> multi dim array

1 // 2D array (matrix) van gehele getallen met 3 rijen en 4 kolommen
2 int[][] matrix = new int[3][4];
3
4 // initialisatie van een 2D array
5 int[][] tabel = {
6     {1, 2, 3, 4},
7     {5, 6, 7, 8},
8     {9, 10, 11, 12}
9 };
10
11 // toegang tot elementen in een 2D array
12 int element = tabel[1][2]; // element in de tweede rij en derde kolom (waarde is 7)
```

```

13
14 // itereren over een 2D array
15 for (int i: tabel) //voor elke rij in tabel
16 {
17     for (int j: matrix[i]) //voor elke kolom in die rij
18     {
19         System.out.println(matrix[i][j]);
20         //output: 1 2 3 4 5 6 7 8 9 10 11 12
21     }
22
23     // je gaat eerst over elke element in de rij en
24     dan naar de volgende rij.
25 }

```

2.6 ArrayLists

een opstelling van een ArrayList:

```

</> arraylist

1 import java.util.ArrayList; //importeer de ArrayList klasse
2
3 // maak een ArrayList van Strings
4
5 private ArrayList<String> namen;
6
7 in constructor:
8 public MijnKlasse() {
9     namen = new ArrayList<String>();
10 }
11
12 // voeg elementen toe aan de ArrayList
13 namen.add("Alice");
14 namen.add("Bob");
15 namen.add("Charlie");
16
17 // toegang tot elementen in de ArrayList
18 String eersteNaam = namen.get(0); // eerste element (index 0)
19
20 // verwijder een element uit de ArrayList
21 namen.remove(1); // verwijdert het element op index 1 ("Bob")
22 namen.remove("Bob"); // verwijdert het element "Bob" als het bestaat
23
24 // grootte van de ArrayList
25 int grootte = namen.size(); // aantal elementen in de ArrayList
26
27 // itereren over een ArrayList
28 for (String naam : namen) {
29     System.out.println(naam);
30     //output: Alice Charlie
31 }
32
33 //arraylists kunnen ook instanties bevatten van
34 andere klassen,
35 bijvoorbeeld een arraylist van personen:
36
37 private ArrayList<Persoon> personen;

```

```

38
39 in constructor:
40     public MijnKlasse() {
41         personen = new ArrayList<Persoon>();
42     }
43
44 // voeg een persoon toe aan de ArrayList
45 public void voegPersoonToe(String naam, int leeftijd)
46 {
47     personen.add(new Persoon(naam, leeftijd)); //persoon(constructor)
48 }
49
50 Je kunt dan ook methoden van de Persoon klasse gebruiken:
51 //scan die personen boven een bepaalde leeftijd verwijderd
52 uit de lijst en die teruggeeft
53 public ArrayList<Persoon> verwijderOuderePersonen(int leeftijdsGrens)
54 {
55     ArrayList<Persoon> verwijderdePersonen = new ArrayList<Persoon>();
56
57     for (Persoon persoon : personen)
58     {
59         //methode van Persoon klasse
60         if (persoon.getLeeftijd() > leeftijdsGrens)
61         {
62             verwijderdePersonen.add(persoon);
63             personen.remove(persoon);
64         }
65     }
66 }

```

3 Class extends

In Java kun je een nieuwe klasse maken die de eigenschappen en methoden van een bestaande klasse overneemt door middel van het sleutelwoord `extends`. Dit wordt ook wel *inheritance* genoemd.

```

</> class extends
1 // Bestaande klasse
2 public class Dier {
3
4     private String naam;
5     private int leeftijd;
6     private String locatie;
7
8     public Dier(String naam, int leeftijd, String locatie) {
9         this.naam = naam;
10        this.leeftijd = leeftijd;
11        this.locatie = locatie;
12    }
13
14    public void getInfo() {
15        System.out.println("Naam: " + naam + ", Leeftijd: " + leeftijd + ", Locatie: " + locatie);
16    }
17 }
18
19 // Nieuwe klasse die Dier uitbreidt
20 public class Hond extends Dier {

```

```

21
22     private String ras;
23
24     public Hond(String naam, int leeftijd, String locatie, String ras) {
25         super(naam, leeftijd, locatie); // roept de constructor van de superklasse Dier aan
26         this.ras = ras;
27     }
28     // je moet methodes niet opnieuw schrijven want een hond is ook een dier
29     public void maakGeluid() {
30         System.out.println("Hond blaft");
31     }
32
33     //alleen hond kan deze methode gebruiken
34     public void speelFetch() {
35         System.out.println("Hond speelt fetch");
36     }
37 }
38
39 // Gebruik van de klassen
40 public class Main {
41     ArrayList<Dier> dieren = new ArrayList<Dier>();
42     dieren.add(new Dier("Leeuw", 5, "Afrika"));
43     dieren.add(new Hond("Buddy", 3, "Thuis", "Labrador"));
44     //je kunt een hond ook toevoegen omdat een hond een dier is
45
46     for (Dier dier : dieren) {
47         dier.getInfo(); // werkt voor zowel Dier als Hond
48         //dier.maakGeluid(); // Fout: Dier heeft deze methode niet
49         if (dier instanceof Hond) {
50             ((Hond) dier).maakGeluid(); // Correct: cast naar Hond om de methode aan te roepen
51         }
52     }
53 }
54
55
56 \section{Hashmaps}
57
58 % TODO: #62 dit kwam op het examen maar geen idee wat het is? Iemand uitleggen?
59
60 \section{Scanner}
61
62 De code om van de Scanner krijg je op het examen maar niet hoe je die gebruikt.
63 \begin{codeblock}[scanner]{java}
64     import java.util.Scanner; //importeer de Scanner klasse
65
66     public class MijnKlasse {
67
68         public MijnKlasse() {
69         }
70
71         public int sumOfTwoNumbers(int a, int b) {
72             return a + b;
73         }
74
75         public void readFromFile(String filename) {
76             try {
77                 Scanner sc = new Scanner(new File(filename));
78                 while(sc.hasNext()) {

```

```

79     String line = sc.nextLine();
80     System.out.println(line);
81 }
82 of met integers:
83 while(sc.hasNextInt()) {
84     int a = sc.nextInt();
85     int b = sc.nextInt();
86     int resultaat = sumOfTwoNumbers(a, b);
87     System.out.println(resultaat);
88 }
89 sc.close();
90 }
91 catch (FileNotFoundException fnfe) {
92     System.out.println("file not found");
93 }
94 }
95
96 //belangrijke methodes van Scanner:
97 hasNext(): controleert of er nog meer tokens zijn
98 next(): leest het volgende token als een String
99 nextInt(): leest het volgende token als een int
100 nextLine(): leest de volgende volledige regel als een String
101 nextFloat(): leest het volgende token als een float
102 nextDouble(): leest het volgende token als een double

```

4 UML diagrammen
